🖐 MediaPipe Gesture Height-Based AprilTag Sorting

1. Course Overview

The function implemented in this lesson can be simply understood as: **use gesture to tell the robot which height block to grasp, robot automatically finds the AprilTag at that height and picks it up**.

The system combines MediaPipe gesture recognition with 3D spatial ranging model. After the user shows a height threshold gesture in front of the camera, the AprilTag detection node estimates the top surface height of each AprilTag block in the field of view in real-time and filters blocks **meeting the height criteria**. When multiple blocks meeting the height criteria exist in the field of view, the system automatically executes **AprilTag priority arbitration mechanism (prioritize grasping ID 1-4 AprilTags in order 1 > 2 > 3 > 4)**, locking onto a single target. The height threshold is controlled by gesture number accumulation (e.g., gesture 1 accumulates threshold, gesture 4 means filtering blocks with height ≥4cm for double-layer stacked blocks), achieving orderly, interference-resistant human-robot interactive dynamic sorting.

> **Note**
>
> **This tutorial uses the accompanying 3×3cm AprilTag cube wooden blocks.**

---

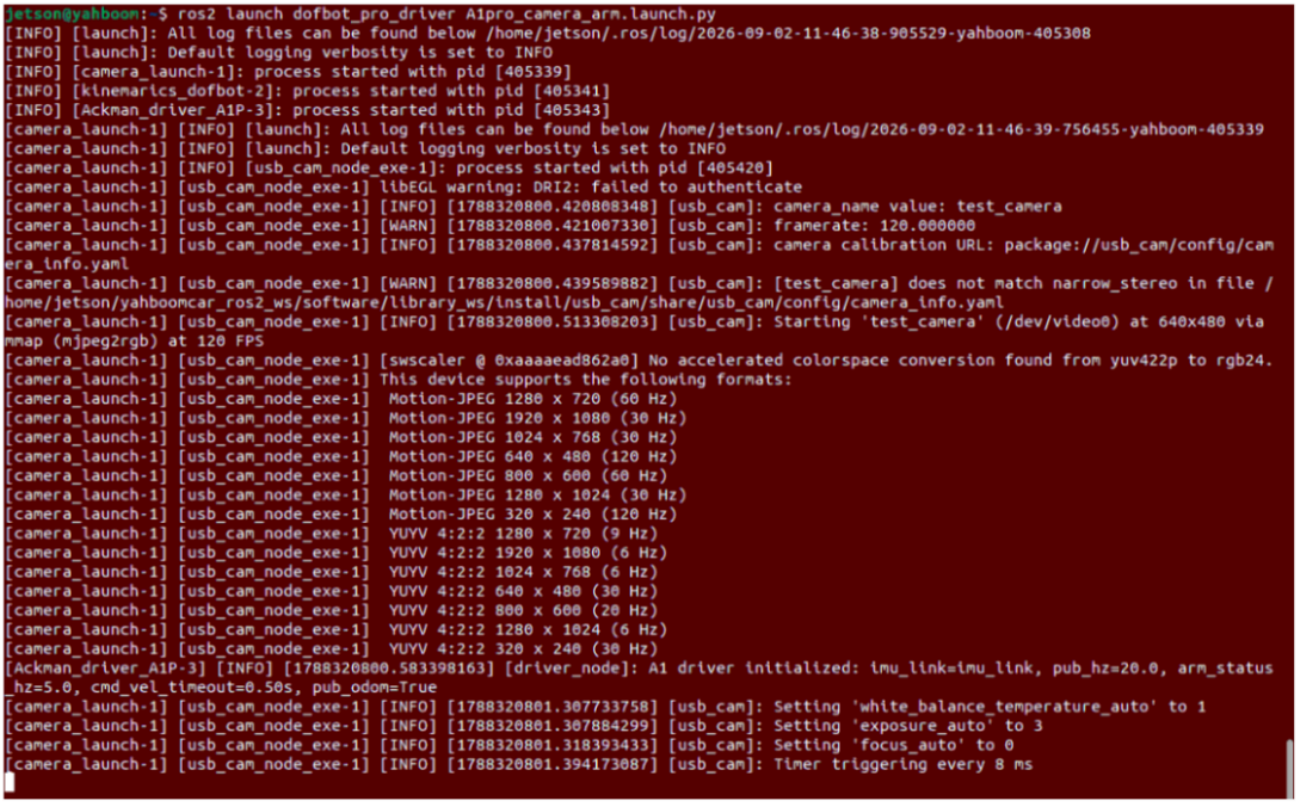
2. Preparation and Running

This case requires opening 4 terminals to start the system sequentially:

2.1 Start Camera Driver and Kinematics Solver Environment

Run in **Terminal 1**:

```
ros2 launch dofbot_pro_driver A1pro_camera_arm.launch.py
```

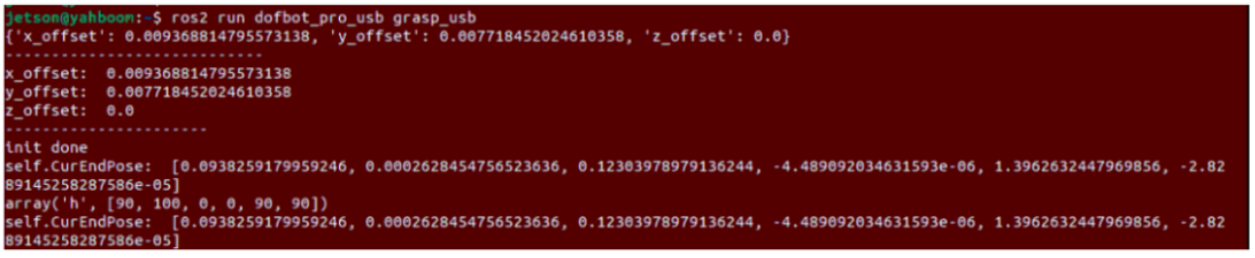


2.2 Start Robotic Arm Desktop Grasping Action Execution Node

Run in **Terminal 2**:

```
# Depth camera version (Nuwa)
ros2 run dofbot_pro_driver grasp_desktop

# Or USB camera version
ros2 run dofbot_pro_usb grasp_usb
```



2.3 Start Gesture Height Sorting Master Node

Run in **Terminal 3**:

```
# Depth camera version (Nuwa)
ros2 run dofbot_pro_driver apriltagHeight_gesture

# Or USB camera version
ros2 run dofbot_pro_usb apriltagHeight_gesture
```

```
jetson@yahboom:~$ ros2 run dofbot_pro_usb apriltagHeight_gesture
init done
---------------------
Loaded Offset -> x: 0.009368814795573138, y: 0.007718452024610358, z: 0.0
Init done.
```

2.4 Start MediaPipe Gesture Recognition Node

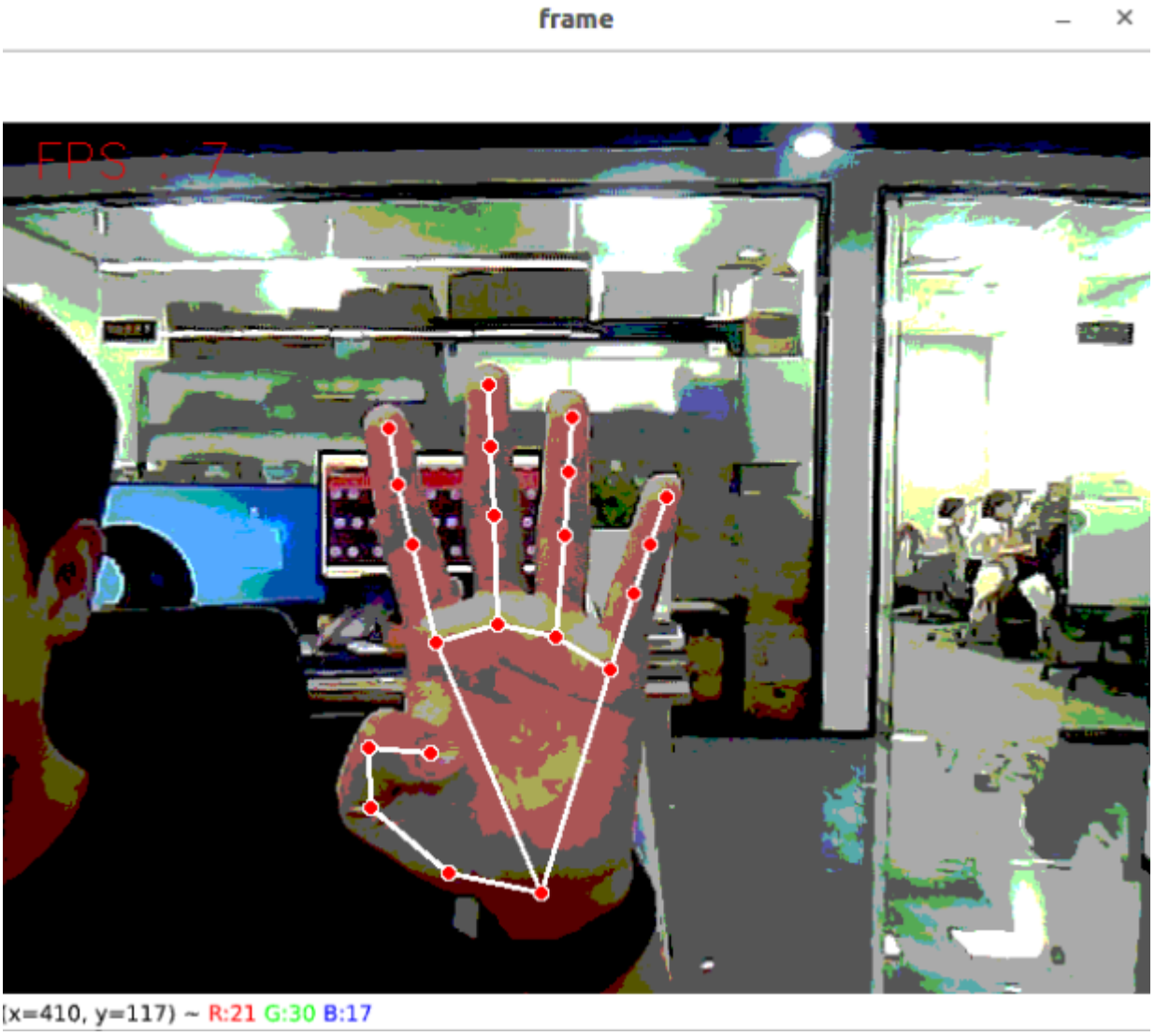
Run in **Terminal 4**:

```
# Depth camera version (Nuwa)
ros2 run dofbot_pro_driver mediapipe_detect

# Or USB camera version
ros2 run dofbot_pro_usb mediapipe_detect
```

```
jetson@yahboom:~$ ros2 run dofbot_pro_usb mediapipe_detect
/home/jetson/.local/lib/python3.10/site-packages/matplotlib/projections/__init__.py:63:
UserWarning: Unable to import Axes3D. This may be due to multiple versions of Matplotlib
 being installed (e.g. as a system package and as a pip package). As a result, the 3D pr
ojection is not available.
  warnings.warn("Unable to import Axes3D. This may be due to multiple versions of "
init done
---------------------
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
sum of fingers:  1
sum of fingers:  1
cnt: 1
```

Show the gesture for height greater than how many centimeters, for example 4:



3. Operation Demo and Interaction Flow

1. **Material preparation**: Place single-layer blocks (height approximately 30mm) and double-layer stacked blocks (height approximately 60mm) together in the working area;
2. **Gesture sets height selection threshold**:
    - Show number gesture in front of the camera to set target height:
        - **Show gesture "1"**: Target height threshold set to above 1cm (grasps all blocks on table);
        - **Show gesture "4"**: Target height threshold accumulates to above 4cm (precisely filters and grasps double-layer stacked over-standard blocks);
3. **Pose smooth switching and detection**:
    - Keep gesture for approximately 1 second, buzzer sounds to confirm gesture;
    - Robotic arm automatically switches from gesture interaction pose to desktop patrol pose;
4. **Autonomous approach and grasping**:
    - System calculates spatial height of all AprilTags in the field of view; if multiple AprilTags are above the set height, the system will **prioritize lock onto ID 1-4 AprilTags (arbitration order 1 > 2 > 3 > 4) and highlight them on screen with yellow circle and** `TARGET ID: x`;

- Chassis smoothly moves forward, robotic arm adaptively calculates gripper rotation angle and descends to grasp;

5. **Reset and wait for next round**:
   - After grasping, front wheel servo straightens, robotic arm lifts back to gesture recognition pose, sends reset signal to gesture node, threshold resets to initial value, waiting for next gesture.

---

4. Core Source Code Analysis

The system is completed by "MediaPipe gesture keypoint computation" and "AprilTag height decision" working in coordination:

Node Function	USB Monocular Camera Solution	3D Depth Camera Solution (Nuwa-HP60C)
Gesture Skeleton Detection	`dofbot_pro_usb/mediapipe_detect.py`	`dofbot_pro_driver/mediapipe_detect.py`
Height Sorting Master	`dofbot_pro_usb/apriltagHeight_gesture.py`	`dofbot_pro_driver/apriltagHeight_gesture.py`
IK Grasping Execution	`dofbot_pro_usb/grasp_usb.py`	`dofbot_pro_driver/grasp_desktop.py`

The main technical implementation comparison:

Comparison Dimension	USB Monocular Camera Solution	3D Depth Camera Solution (Nuwa-HP60C)
Height Calculation Principle	Pinhole calculates depth, derive block height by subtracting measured depth from calibrated reference distance	Depth matrix directly indexes centroid physical distance, combine with forward kinematics to solve world frame Z-axis absolute height
Threshold Accumulation Mechanism	Gesture ID accumulates to `Target_height`	Gesture ID accumulates to `Target_height`
Chassis Working Distance	Chassis PID control advances to approximately 190mm	Chassis PID control advances to approximately 205mm (`cen_dist = 205`)
Spatial Calibration Compensation	Superimpose X, Y 2D plane calibration deviation	Synchronously superimpose X, Y, Z 3D omnidirectional calibration compensation (`offset_value.yaml`)
Gripper Surface Adaptive	Default fixed-angle grasping	Combine AprilTag corner vectors, adaptively rotate via `compute_joint5`

4.1 Gesture Number Dynamic Accumulation Height Threshold

Regardless of version, when the gesture callback function receives gesture message, it accumulates the number to the target height property:

```
# apriltagHeight_gesture.py -- Gesture height accumulation
def get_GesturetIdCallBack(self, msg):
    self.Target_height = msg.data + self.Target_height  # Accumulate height threshold
    self.pubSix_Arm(self.init_joints)                   # Switch to desktop patrol pose
    self.start_time = time.time()
    self.count = True                                   # Enable stable delay
    self.pubPos_flag = False
```

4.2 Two Camera Solutions' Height Calculation and Target Filtering Implementation

1. USB Monocular Camera Solution (`apriltagHeight_gesture.py`)

The monocular camera uses pinhole imaging to calculate tag depth `cam_z`, then subtracts it from the calibrated reference depth from camera to table (`0.255m`) to get centimeter-level height:

```
# apriltagHeight_gesture.py (USB) -- Pinhole relative height calculation and filtering
cam_coords = self.compute_depth_pinhole(tag.corners, fx, fy)
cam_z = float(cam_coords[2])
height_cm = (0.255 - cam_z) * 100  # Relative table height

# Only process blocks with height greater than or equal to set threshold
if height_cm >= self.Target_height:
    # Lock target and start chassis approach
```

2. 3D Depth Camera Solution (`apriltagHeight_gesture.py`)

The depth camera directly extracts the true physical distance at the tag center from the aligned depth map slice, combines with forward kinematics service to solve world reference coordinates `pose`. `pose[2] * 100` is the precise physical height:

```
# apriltagHeight_gesture.py (Depth Camera) -- World coordinate absolute height computation
and filtering
cz = depth_image[int(tag.center[1]), int(tag.center[0])] / 1000.0  # True physical depth
(meters)
if cz != 0:
    pose = self.compute_pose(tag.center[0], tag.center[1], cz)
    height_cm = pose[2] * 100  # Base absolute height (centimeters)

    # Height meets criteria and delay is ready, execute grasping flow
    if height_cm >= self.Target_height:
        dist_detect = math.hypot(pose[0], pose[1]) * 1000.0
        if abs(dist_detect - self.cen_dist) > 5.0 and self.adjust_dist:
            self.move_dist(dist_detect, tag.center[0])
```

3. Grasping and Closed-Loop Reset

After chassis is in position, drive joint 5 deflection alignment, publish pose to trigger grasping. After grasping completes, execute front wheel servo straightening action, lift robotic arm back to gesture recognition pose (`identify_joints`), and publish reset signal to `reset_gesture`. Threshold returns to default initial value, achieving fully autonomous cyclic sorting.

4.3 Grasping Offset Tuning (Both Camera Models)

If position deviation occurs during actual grasping (such as missing, knocking over objects, offset left/right, or hitting the table), you can directly modify the spatial offset configuration file for the corresponding camera:

1. Where to Modify

Configuration files are located in the `~/yahboomcar_ros2_ws/dofbot_pro_ws/src/dofbot_pro_driver/config/` directory. Select based on camera type:

- **3D Depth Camera (Nuwa-HP60C)**: `offset_value.yaml`
- **USB Camera**: `usb_offset_value.yaml`

2. How to Modify

The file contains offset parameters in X, Y, Z directions (unit: meters, recommended adjustment per tweak is `0.005`~`0.01`, i.e., 5~10mm):

- `x_offset` **(forward/backward direction, positive toward vehicle front)**:
    - Gripper extends too far and knocks over object **decrease value** (adjust smaller or set negative)
    - Gripper doesn't extend far enough and misses **increase value** (adjust larger)
- `y_offset` **(left/right direction, positive toward vehicle left)**:
    - Gripper deviates right **increase value** (correct left)
    - Gripper deviates left **decrease value** (correct right)
- `z_offset` **(up/down height, positive toward ceiling)**:
    - Gripper too high in air and can't grasp **decrease value** (probe deeper)
    - Gripper hits or scrapes table **increase value** (raise higher)